AgenticBot: LangGraph and LangChain Extension

An extension to AgenticBot.md. It covers only what changes if the conversation engine is built on LangGraph and LangChain. Everything not listed here is as in that plan: the requirements, data structuring, LiveKit and SIP telephony, STT and TTS, the knowledge base, safety rules, targets and the section numbering.

1. What changes

Main plan	With this extension
Workflow engine (4.4, E5): own YAML-driven engine	One LangGraph subgraph per workflow
Conversation state (E2, section 5): Postgres session table	LangGraph checkpointer in Postgres
Router (E4, 8.1): a router module	A router node in the top-level graph
RAG (E16, 7.4): plain-code module	A retrieval node; LangChain utilities optional
LiveKit link: engine called directly	LiveKit's LangChain plugin wraps the graph as the agent's LLM
Adding a workflow (14, E17): YAML only	A builder function turns the YAML into a standard subgraph
Phase 4	A latency spike, then a checkpointer phase

2. How it works

- **Subgraph per workflow.** A top-level graph has a router node that sends each turn to a workflow subgraph, the FAQ and lookup node, the RAG node or human handoff.
- **State.** A checkpointer saves state per call (thread id = call id) in Postgres (checkpointers). Interrupts handle confirmations and need the checkpointer (interrupts).
- **LiveKit link.** The plugin `livekit-plugins-langchain` has an `LLMAdapter` that wraps the compiled graph so it acts as the agent's LLM (docs). The graph's output becomes the spoken text, so reply nodes emit fixed, templated Roman Urdu. The plugin supports graph-based agents only, and streams token by token by default.
- **LangChain is optional.** It supplies text splitters, the embedding wrapper and the `pgvector` connector for RAG. Plain code can replace any of them.
- **What we still write.** LangGraph does not provide the workflow stack rules (limit of two suspended workflows, shared facts), digression handling, slot-filling helpers, normalizers, or verification.

3. Tech stack changes (section 5)

These rows replace the four engine rows of the main plan's tech stack. All other rows are unchanged.

Layer	Technology	Used to build
Workflow orchestration	LangGraph	One subgraph per workflow, a router node, interrupts for confirmations
LiveKit link	`livekit-plugins-langchain` `LLMAdapter`	Using the compiled graph as the agent's LLM
Conversation state	LangGraph checkpointer in Postgres, keyed by call id	Saved workflow state, suspend and resume
RAG utilities	LangChain text splitters, embedding wrapper, `pgvector` connector (optional)	Chunking and vector-store access

4. Module changes (section 6)

#	Module	Change
E2	Conversation state	LangGraph checkpointer in Postgres keyed by call id, so a restart does not drop calls; workflow stack and shared facts kept in graph state
E4	Router	A router node in the top graph; still rules first, then Qwen3-8B; build the held-out real-utterance set as before
E5	Workflow subgraphs	One subgraph per workflow, with our stack and digression rules; rewrite the prototype workflows as subgraphs and port the slot and normalizer logic
E16	Retrieval (RAG)	A retrieval node with entity and validity filters; LangChain splitters optional
E17	Registry and scaffolding	Add a subgraph builder (see section 6 below)

5. Switching on the graph (section 8.1)

The top-level graph's state holds the active workflow, the suspended stack, the shared facts and the pending question. Each workflow's slots are stored in this parent state, so resume does not depend on per-subgraph checkpoints. The router node applies the same policy as in the main plan and routes to the target subgraph. The checkpointer persists the state. Whether suspending and resuming workflows this way works cleanly in LangGraph is checked in the phase 4 spike.

6. Adding a workflow (section 14)

A workflow is a subgraph. A builder function creates the standard form-like subgraph (collect slots, read back, confirm, call one API) from the workflow's YAML definition. The registry, manifest, router shortlist, `check_workflow` and guardrails are as in section 14 of the main plan. The builder is code we write and maintain.

7. Phases (section 13)

Phases 1 to 3 are as in the main plan. Replace phase 4 and add a phase 5; the main plan's phases 5 to 10 become 6 to 11.

Phase	Work	Gate
4	**Latency spike.** Build `home_sampling`, the router and the RAG node as a LangGraph graph over LiveKit with the real endpoints. Measure scenario pass rate and P95 time to the first spoken sentence.	Meets the latency target and passes the scenarios; otherwise use the main plan's engine
5	Conversation state in the checkpointer; build the registry, the subgraph builder, scaffold and `check_workflow`; rebuild the existing two workflows as subgraphs on the real structured data	Scenarios pass; state survives a restart

8. Conditions, caveats and risks

Conditions - Every API call and submit stays in deterministic code nodes. The LLM only extracts names and helps routing. - Pin the LangGraph, LangChain and LiveKit plugin versions.

To verify in the phase 4 spike - Turn latency against the 3 s target. - Whether token streaming works when a node wraps a compiled agent; use subgraphs. - How LiveKit's chat context and the checkpointer interact each turn. - How call control (transfer, hangup, SMS) is triggered from graph nodes through a service they can call. - That the Postgres checkpointer is available at the pinned version.

LiveKit's own tasks and handoffs are still not used for the workflows.

9. Decision and fallback

Adopt this extension only if the phase 4 spike meets the latency target and the scenarios pass. Otherwise build the main plan's engine unchanged.

10. Sources

- LiveKit LangChain plugin: https://docs.livekit.io/agents/models/llm/langchain
- LiveKit, moving LangChain workflows to voice: https://livekit.com/blog/langchain-to-livekit
- LangGraph checkpointers: https://docs.langchain.com/oss/langgraph/checkpointers
- LangGraph interrupts: https://docs.langchain.com/oss/langgraph/interrupts